Git Basics: a hands-on introduction

Laura Prieto Saavedra
Scientist and Research Software Engineer
This work is licensed under a CC BY 4.0 license.

April 02, 2026, Software and Data Seminar

Context

Why is this relevant for you?

- Do you share or collaborate at work?
- How do you keep track of your papers, scripts and data?
- Useful for every researcher who wishes to have a clear workflow.

Goal: to learn all the basic commands of Git and find settings that work for you on your laptop.

How? This session will be **interactive**. I will present the concepts and you will be able to try them out in your computer using a sample directory.

Note: the material is based on previous seminars, CodeRefinery lessons and the Git Pro book available online `https://git-scm.com/book/en/v2`.

Prerequisites: what do you need?

A laptop with internet access and a (`bash`) terminal:

- For most Linux distributions: `Ctrl+t`.
- For macOS: `Cmd` + `space` and type “Terminal".
- For Windows users: WSL2 or PuTTy recommended.

Git installation: verify it is installed by typing in your terminal `git --version`. If there is no Git, install it:

- For most Linux distributions: `sudo apt update && sudo apt install git`.
- For macOS: open terminal and type `brew install git`.
- For Windows users: the same as Linux in their WSL terminal.

Materials: download sample directory from the seminar chat.

During the presentation:

- Warnings will be shown in red.
- Best practices will be shown in orange.
- Tasks for you will be shown in blue.

Content

1. Introduction
2. Basics
3. First commands
4. Commit history
5. Branching
6. What comes next?

Content

1. Introduction
2. Basics
3. First commands
4. Commit history
5. Branching
6. What comes next?

What is version control?

Definition: Version control is a system that records changes to a file or set of files over time so that you can recall specific versions later.

A (very) brief history of Git:

- 2002: the Linux kernel project began using proprietary VC software.
- 2005: the community started too develop their own tool Git due to problems with that software.

Main qualities:

- Extremely fast.
- Efficient for large projects.
- Simple design.
- Support for non-linear development.

This sounds useful for software development, is it useful for us as researchers?

Why is it relevant for us?

Useful for:

- a variety of files:
  - A research article
  - Other documents: proofs, abstracts, proposals, personal research log, ...
  - Website source files
  - Teaching material
  - Code: complete programs, scripts, ...
- if working:
  - alone
  - in a team or with collaborators

→ you can keep track of different versions and STOP fearing data loss.

Warning: not all data extensions are supported, e.g., Word files

So is Git the same as GitHub or GitLab?

Git vs Git-hosting service

While Git is the command-line program you install in your laptop, you need a Git-hosting service to store it in the web (or internally).

Options:

- GitHub
- GitLab
- BitBucket
- Codeberg (forgejo)
- ...

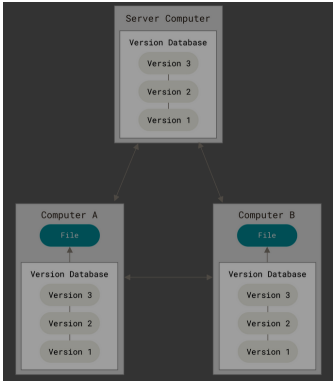
The Git-hosting service stores one or more Git repositories and provides a User Interface (UI) plus extra collaboration features. Best practices: use internal WIAS GitLab if possible.

Content

1. Introduction
2. Basics
3. First commands
4. Commit history
5. Branching
6. What comes next?

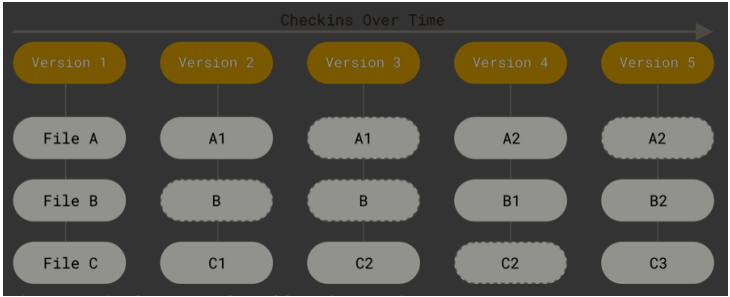
How does it work?

- There is a server with snapshots of the files that fully mirror the **repository** (directory).
- It tracks changes to files and directories over time.
- You can have a local copy of the files on your computer and/or several computers.
- Other members of your team can also have a copy.
- All of you can contribute to the same project and share your changes with each other.



A simple but important concept...

Git stores snapshots of your files, not a list of changes.

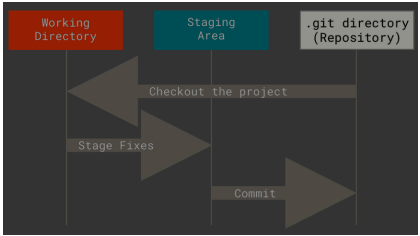


This idea makes Git extremely powerful. It is impossible to change the contents of any file or directory without Git knowing about it. Git assigns a unique identifier to each snapshot, also known as a hash, e.g., 24b9da6552252987aa493b52f8696cd6d3b00373.

Workflow

The general Git workflow consists in:

- You modify files in your working tree.
- You **selectively** stage those changes you want to be part of your next commit.
- You commit, which takes those files and stores a snapshot permanently to your Git directory.



Sample directory

In this session we will be working with a sample directory **"sample_repository"**.

The directory includes three main folders:

- **article**: a `.tex` document of a WIAS preprint with figures and references.
- **data:** a `.csv` file with the data extracted from a simple PINNs code.
- **scripts:** two `.py` scripts that allow to generate the figures for the article by reading the data.

Task: inspect the directory in the terminal by using basic commands. You can also visualize the files using an editor of your choice, e.g., VSCode.

Basic terminal commands:

- `cd`: to go into a directory.
- `ls`: to list the contents in the current directory.
- `cd ..`: to go to the previous directory.

Content

1. Introduction
2. Basics
3. First commands
4. Commit history
5. Branching
6. What comes next?

Creating a Git repository

There are two ways of having a Git repository:

Command 1: `git init`

Command 2: `git clone <url>`

Remarks:

- Command 1 allows you to take a local directory and turn it into a Git repository.
- Command 2 clones (copies) a Git repository stored on a Git-hosting service.

Task: use command 1 to initialize the local sample directory.

What happens behind the scenes?

A `.git` subdirectory will be created and contains all the necessary things for Git to track your project. You can use the command `ls -a` to check that the folder was created.

We will differ the usage of command 2 to the next session.

Configuring Git for the first time

There are certain things that you should set up before using Git on your computer. All these settings come in a tool called `git config`.

Command 3: set your name: `git config user.name "Your Name"`

Command 4: set your email: `git config user.email "lastname@wias-berlin.de"`

- You can add the `--global` flag to set it for ALL your repositories.

Best practices: Always ensure that you have the correct information to ensure that your commits are properly attributed to you.

Command 5: set your editor: `git config core.editor nano`

You can choose `nano`, `vim`, etc. To verify your information use `git config --list`.

Task: set up your Git identity and editor in the sample repository.

Checking the status of your repository

The most important command in Git is the one that allows us to check the status of the repository:

Command 6: `git status`

For a new repository, you will have the following output:

```
On branch master
No commits yet
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        article/
        data/
        scripts/
nothing added to commit but untracked files present (use "git add" to track)
```

Task: test the command in your local repository.

Adding something to your repository

Currently, our directory is not tracking any file, so let us add all the untracked files using the following command:

Command 7: `git add .`

- We can also add one file at the time by replacing `.` with the complete path.

Task: add all the untracked files and check the status of the repository.

The next step is to commit the changes:

Command 8: `git commit -m "Meaningful message"`

- Without the flag `-m` and the message, it will open your editor to write the message.
- Best practices: the message should start by a verb, be concise and descriptive.

Task: commit the previously staged files and check the status of the repository.

Recording changes and check what changed in existing files

Remember the workflow: modify, stage the changes and commit.

Task: replace the name of the first author of the paper in the `article/main.tex` file by your name. Change something in the `article/references.bib` file. Verify the status of the repository.

Command 9: `git diff path/to/file`

- You can omit the path and it will print all the unstaged changes.
- You can add the flag “-- staged" for only staged files.

To discard changes to a particular file and come back to the previous version:

Command 10: `git restore path/to/file`

Warning: use with caution as you will not be able to recover those changes.

Task: restore the references.bib file, and add and commit the changes to the article.

Viewing the commit history

Once you have created several commits, you will probably want to check what has happened in your project.

For this, the following command is used:

Command 11: `git log`

- It will list the commits made in that repository in reverse chronological order.
- It contains the SHA-1 checksum, the author's name and email, the date written and the commit message.
- A short version can be obtained by adding the flag `--oneline`.
- A graphical version can be obtained by adding the flag `--graph`.
- You can add the path to a specific file to see the commits that modified it.
- You can add the `-- stat` for more details.

Task: test all the different versions in your repository.

Undoing things

Situation 1: you committed but realize you made a mistake or forgot some changes.

Command 12: `git commit --amend`

- Useful to fix your last commit.
- Best practices: only use it if your changes are still local and have not been pushed yet (we will talk about this in the next session).

Situation 2: you have added all the files instead of one, what to do?

Command 13: `git restore --staged path/to/file`

- This will unstage the changes but you will not lose them.

Task: try them out in your local repository.

Moving and removing files

There are specific commands to move or remove files from your repository. To rename (move) a file:

Command 14: `git mv old_name new_name`

- It moves the file and stages the rename in a clean way.

To delete a file:

Command 15: `git rm path/to/file`

- It deletes the file and stages the deletion in a clean way.
- To stop tracking a file without deleting add the flag `-- cached`.

Best practices: always use these commands instead of moving and removing files from the editor or the command line to keep a clean history and prevent data loss.

Task: rename one of the plot scripts in the repository.

Some additional tips when working with repositories

Best practices: ignoring files

Often we have files in the repository that you do not want Git to automatically add or even show you as being untracked. For this, we can create a **".gitignore"** file with a list of files to be ignored.

Task: Follow the example:

1. Let us create a folder results `mkdir results`.
2. Let us create several files with certain format.
3. Add them to the “.gitignore" file in several ways.

Best practices: Using a `README.md` file to describe your repository

This file should be present in every repository. It should briefly describe: the purpose of the repository, the contents, how to use the contents and a license if needed.

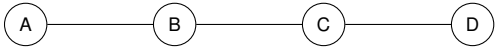
Task: add a README.md file to your repository.

Content

1. Introduction
2. Basics
3. First commands
4. Commit history
5. Branching
6. What comes next?

Why are commits useful?

A commit is a snapshot of your project at a specific moment in time. We can think of Git as a timeline of our project:



This is powerful because it allows us to:

- Restore deleted code.
- Compare versions.
- Find bugs (using `git bisect`).

To show difference between a commit and the current commit:

Command 16: `git show commit_id`

- Displays the exact changes and the metadata.

Task: try command 16 by replacing the commit_id by the hash.

Check differences between commits

To check the difference between two specific commits:

Command 17: `git diff commit_id_1 commit_id_2`

To check the changes to one file since certain commit:

Command 18: `git diff commit_id /path/to/file`

To navigate between commits safely:

Command 19: `git checkout commit_id`

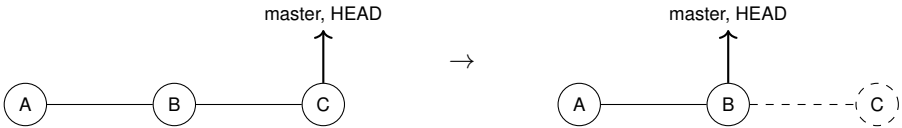
- To go back to the last commit: `git switch -`.

Task: try any of those commands with your current history.

Undoing the most recent commit

This command allow us to rewrite history:

Command 20: `git reset HEAD^`



Warning: use with caution as you might lose important work.

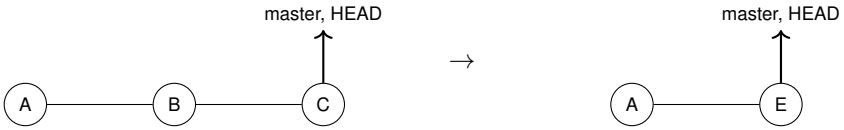
Best practices: only use if the branch is local and nobody else is working based on it.

Task: make one commit and delete it using this command.

Squashing certain number of commits into one

This command allow us to rewrite history:

Command 21: `git rebase -i HEAD~2`



Warning: use with caution as you might lose important work.

Best practices: Only use if the branch is local and nobody else is working based on it.

Task: try it out with two commits.

Checking a version of a previous file in certain commit

Sometimes we want to check the version of one of the files in a previous commit. For this we can use the following command:

Command 22: `git show commit_id:path/to/file`

- You can use the normal pipeline terminal command to save it to a new file of your choice as follows: `git show commit_id:path/to/file > copy_of_file`.

An alternative to this command would be to go back to the commit and check the file directly.

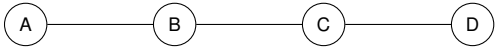
Task: try it out with one of the files and the initial commit.

Content

1. Introduction
2. Basics
3. First commands
4. Commit history
5. Branching
6. What comes next?

Branches

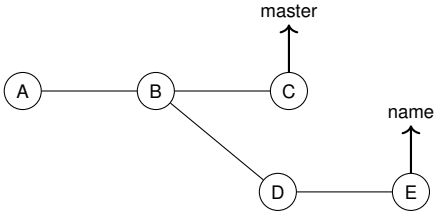
So far we have been working on a branch called `master` (see the git status output) and our history has been linear.



Branches allow us to work on **different timelines at the same time**.

This is useful because it allows:

- safe experimentation.
- parallel development.
- collaboration workflow.
- easy rollback.



How do they work?

Initial commands to work with branches

To see the list of all branches:

Command 23: `git branch`

- So far, we only have a branch called master.

To create a new branch:

Command 24: `git branch name`

To switch to the new branch:

Command 25: `git switch name`

- Alternative: `git checkout name`
- One can also create and switch at the same time using `git switch -c name` or `git checkout -b name`.

Task: create a new branch and switch to it. Change something, commit and inspect history.

Deleting branches

To delete a branch:

Command 26: `git branch -d name`

- This will only work if the content of the branch has already been merged (coming soon).

To force delete a branch:

Command 27: `git branch -D name`

- Warning: use with caution, once deleted it is gone forever.

Tip: an IDE might be very useful when trying to visualize the different branches.

- Branches are extremely powerful and useful also when working in small teams or individually.

Task: create a new branch and delete it.

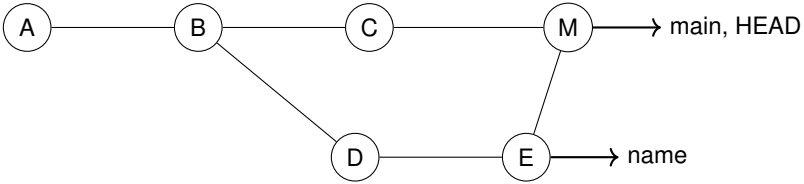
Combining branches

Scenario: we created a new branch experimented with some changes and now we want to add this changes to my master branch.

Option 1: combine with merge

Switch to the main branch and merge the new branch:

Command 28: `git switch master` and `git merge name`



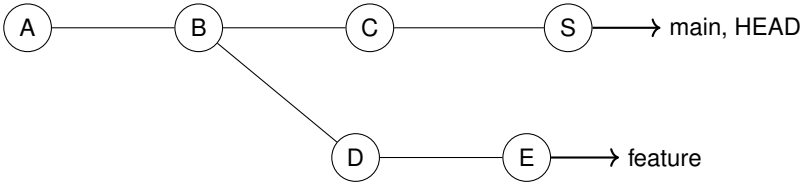
- Preserves D and E in history and creates a new M commit combining the changes.

Combining branches

Option 2: combine with squash merge

Switch to the main branch and merge the new branch:

Command 29: `git switch master`, followed by `git merge --squash name` and `git commit -m`



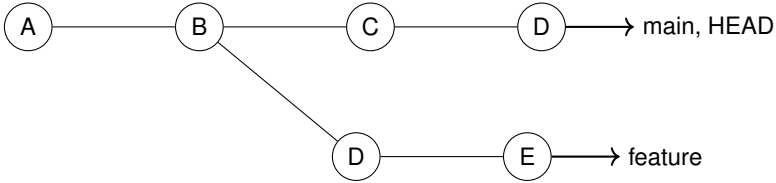
- All the individual commits in the new branch are now replaced by one single commit S in the master branch.

Combining branches

Option 3: copy one commit into the current branch

Switch to the master branch and add one commit of the new branch:

Command 30. `git cherry-pick commit`

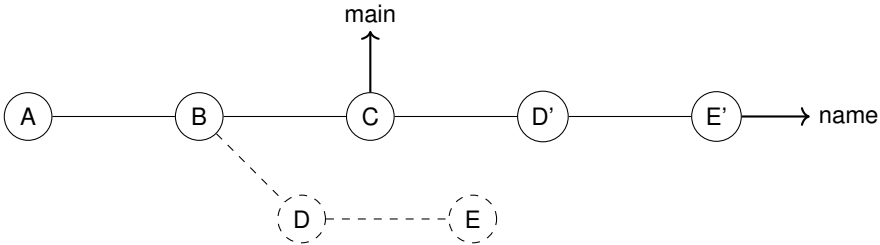


Task: try any of the previous commands used to combine branches.

Combining branches

Scenario: we created a new branch experimented with some changes and now the master branch has some updates that we want to add to the new branch. Switch to the new branch and rebase it:

Command 31. `git switch name` and `git rebase master`



- The new branch has all the commits of master plus the new ones on the top.

Conflict resolution

Git is very good at handling unrelated changes within the same file. However, what if we change something in exactly the same line? **Git needs your help to figure out what to keep!**

Task: create a new branch and change the title of the solution plot. At the same time go back to master and change the title to a different thing.

1. Use `git diff master new_branch` and see that the same line was changed.
2. Try to merge the new_branch into master using `git switch master` and `git merge new_branch`.
3. You should get a merge conflict:

```
Auto-merging scripts/plot_solutions.py
CONFLICT (content): Merge conflict in scripts/plot_solutions.py
Automatic merge failed; fix conflicts and then commit the result.
```

What to do?

Conflict resolution

Option 1: fix it on the terminal.

1. Open the file using an editor.
2. Edit fix between the conflict resolution markers <<<<< and >>>>>.
3. Add the file.
4. Commit with meaningful message.

Option 2: resolve it using the IDE interface.

1. Open file which will be marked with different colors.
2. Accept the change you want to keep, add the file and commit.

Best practices: the first option is error prone and useful for small changes. For larger changes use an IDE, it makes your life so much easier.

Task: try to solve the conflict using one of those options.

- While this may seem trivial in this example, when collaborating with others, you will often have to resolve conflicts. Do not feel overwhelmed, you will gain confidence as you go.

Content

1. Introduction
2. Basics
3. First commands
4. Commit history
5. Branching
6. What comes next?

Final remarks

To conclude:

- While Git has an initial steep learning curve, it can be very useful for your research projects.
- Version control can be used for almost every type of data and/or software.
- Reach out if you want to learn something more specific.

Thank you for your attention!

Software and Data Seminar

References

Pro Git book.
https://git-scm.com/book/en/v2.
[Accessed 01-04-2026].

Bast, R., Wikfeldt, K. T., Darst, R., Razick, S., Iusan, D., Lindi, B., Andrei, C.-O., Suvilehto, J., and Fasshauer, E. (2025).
Introduction to version control with git - CodeRefinery lesson.